

AutoE2E: Don't speak – language free end-to-end autonomy

A Real-Time Driving Model for L2++ ADAS and L4 Robotaxis

Autoware Foundation Robotaxi Working Group[1]

Abstract

AutoE2E is a fully end-to-end driving policy – a vision transformer trainable with a single backpropagation step, built entirely by the Autoware Foundation Robotaxi Working Group's own contributors rather than a frontier AI lab – targeting Level 2++ passenger-vehicle and Level 4 robotaxi applications, and released as the reactive band of a larger open-source autonomous-driving stack whose world-action band is designed around a JEPA-style, non-linguistic future-feature objective rather than a language-model reasoning trace (Sec. 2). Unlike end-to-end planners that condition on a coarse directional token, AutoE2E consumes the full HD map through a native Lanelet2 raster, compatible out of the box with the Autoware ecosystem. At 79.9 M total parameters (1.45 M trainable) and 101 ms inference on commodity GPU hardware, it is roughly two orders of magnitude smaller than the vision-language-action planners it is benchmarked against – reported by the working group at approximately 4 B (Qwen-Drive-1.0) and 10 B (Alpamayo) parameters. On KITScenes Val, AutoE2E's fine-tuned checkpoint reports the lowest mean error at every horizon through 3 s among eight compared open-weights driving policies, and remains within the top two at 5 s. Training data, infrastructure, the released DataModelConsole dashboard, and a preliminary real-vehicle result from TU Munich's EDGAR platform are documented throughout. This is an early research release: evaluation is open-loop only, AutoE2E is not a closed-loop policy, and no statistical significance or causal navigation-input gain is claimed.

Code and release: `https://github.com/autowarefoundation/auto_e2e/releases/tag/v1.0` (Apache-2.0)
Checkpoints: `https://huggingface.co/AutowareFoundation/auto_e2e` (`kitscenes-epoch-5`, fine-tuned; `nuplan-epoch-5`, baseline – weight license follows the source training datasets, not Apache-2.0; see Sec. 10)
Dashboard: `https://d2itskdqq39tx1.cloudfront.net/` (DataModelConsole, Sec. 4)
Keywords. end-to-end autonomous driving, temporal BEV, HD-map conditioning, route conditioning, open-loop evaluation, real-vehicle validation, KITScenes, nuPlan, open-source

Contents

1 Introduction **2**
1.1 Related Work . . . 3

2 Model Architecture **4**
2.1 Deployment Contract . . . 4
2.2 Phase 1: Temporal Camera BEV Baseline . . . 5
2.3 Phase 2: Map- and Route-Conditioned Extensions . . . 6
2.4 Control Planner . . . 10
2.5 Model Size . . . 10

3 Training **11**
3.1 Datasets . . . 11
3.2 How It Was Trained . . . 12
3.3 Where It Was Trained . . . 12

4 The DataModelConsole Deliverable **13**

[1] A full contributor list is given in Sec. 9.

5 Benchmarks **14**
5.1 Population and Protocol . . . 14
5.2 Models Used for Comparison . . . 14
5.3 Primary Evaluation: KITScenes Val . . . 14
5.4 Secondary Evaluation: KITScenes Test (Camera Only) . . . 15
5.5 Cross-Model Benchmark . . . 16
5.6 Qualitative Examples . . . 17
5.7 Inference Cost and Real-Vehicle Evidence . . . 17

6 Limitations **17**

7 Future Work, Outlook and Collaboration **18**

8 Collaborations **18**

9 Contributors and Acknowledgments **19**
9.1 Project Leads . . . 19
9.2 Core Contributors . . . 19
9.3 Other Contributions . . . 19

10 Reproducibility **19**

1 Introduction

The Autoware Foundation's Robotaxi Working Group is developing an open-source, three-band architecture for autonomous driving (Fig. 1): a reactive band that maps sensing directly to control, a reasoning band, and a world-action band. The three bands are intended to be interfaced together once each is mature, not trained as one monolithic network; development starts with the reactive band, the one expected to change fastest, and is designed so that reasoning and world-action components can be interfaced onto it as they come online. AutoE2E is that reactive band. It is designed to be compact and inspectable rather than to lead on raw capability: a deterministic policy – it outputs one trajectory per input, with no sampling at inference, unlike the generative vision-language-action (VLA) planners it is benchmarked against – with 79.9 M total parameters (1.45 M trainable) that runs at 101 ms on commodity GPU hardware, orders of magnitude smaller and faster than those planners.

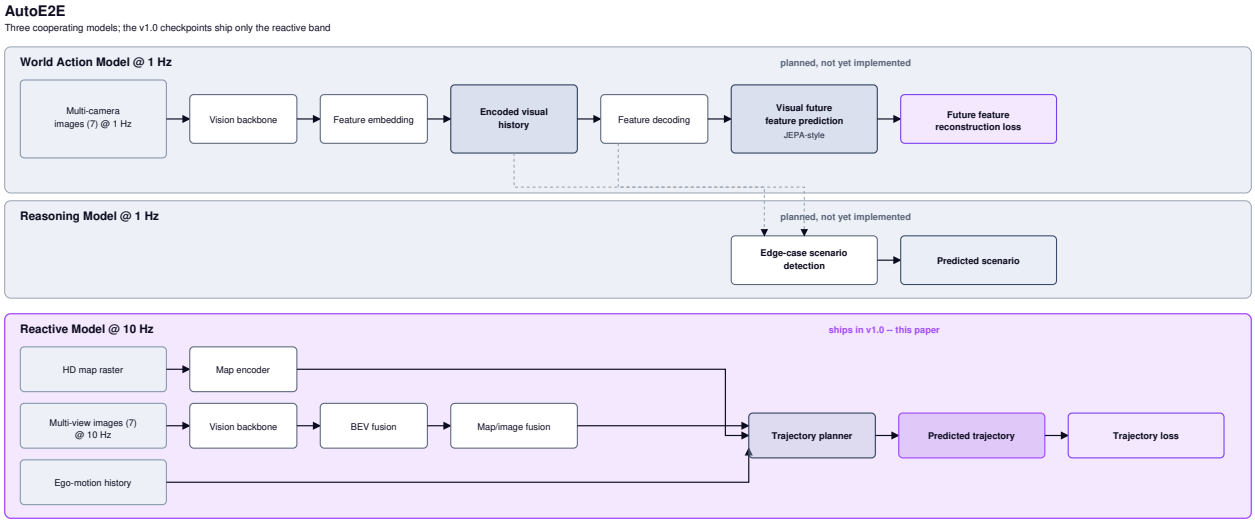


Figure 1: The working group's planned three-band architecture. The reactive model at 10 Hz (violet band) is AutoE2E, the subject of this paper, already implemented and evaluated here. The world-action model at 1 Hz and the reasoning model at 1 Hz (grey bands) are planned interfaces, not yet implemented or trained; no result in this paper depends on them. The world-action band's objective is explicitly non-linguistic: a JEPA-style future-feature reconstruction loss in the camera's own embedding space, not a language-model reasoning trace – the design choice the paper's title refers to.

A reasoning trace is inspectable in a way a raw trajectory is not, but only if the trace is shown to correspond to the action actually taken – a check the VLA literature mostly assumes rather than runs. VLA planners couple a vision encoder and a language-model backbone to produce trajectories together with a reasoning trace, typically through a three-stage recipe of frozen-encoder projector pretraining, supervised fine-tuning on driving data with teacher-generated chain-of-thought annotations, and a reinforcement post-training stage that aligns the policy with closed-loop or preference reward. Representative systems in this family include Qwen-Drive-1.0 [1], which unifies 3D perception, driving VQA, and motion planning on a pretrained vision-language model while leaving its weights untouched, and Alpamayo [2], built on a reasoning-focused backbone and intended primarily as a distillation teacher; the common bottleneck across the family is the gap between research-scale capability and on-vehicle inference budget, addressed almost universally by teacher-student distillation rather than by training a deployable model directly. AutoE2E sits at the opposite end of this design space: a deterministic, non-generative model with a fixed camera-BEV backbone, no language component, and no sampling at inference – and where the working group's roadmap does add a future-prediction capability (Sec. 7), it is specified as a JEPA-style latent-feature objective, predicting the camera pathway's own future embeddings, rather than a token sequence a language model decodes. What AutoE2E gains is a system whose every claim in this paper can be checked against source code and a fixed input/output contract rather than against a reasoning trace whose faithfulness is unverified; what it gives up is the open-vocabulary instruction following and explicit reasoning output the VLA family offers.

One methodological commitment runs through this paper and is worth stating up front. End-to-end (E2E) driving research has developed largely around camera-only open-loop evaluation on nuScenes [11]; UniAD [3] and VAD [7] demonstrated planning-oriented integrated stacks in this setting, with temporal BEV representations such as BEVFormer [4] providing a shared basis. Deployed vehicle policies, however, often receive more than cameras. AutoE2E's deployment contract supplies an HD-map raster and a selected-route raster at every step in addition to camera images. Evaluating this policy only on a camera-only benchmark reports behavior under a condition that omits information the policy was designed to consume; reporting only the map-and-route condition would conversely hide behavior when navigation inputs are unavailable. These evaluations answer different questions and are kept separate throughout this paper.

1.1 Related Work

End-to-end driving policies share a common design space – how much of the perception stack to fold into one network, whether language is part of the model, how temporal information is represented, how a map or route reaches the policy, and how the result is evaluated – and AutoE2E's choices in that space only mean something read against it.

Planning-oriented end-to-end driving. UniAD [3] links detection, tracking, mapping, motion forecasting, occupancy prediction, and planning through shared query interaction, so that every task contributes to the final plan. VAD [7] keeps the same chain but replaces the raster scene with a vectorized one to cut its computational cost, and TransFuser [8][9] fuses camera and LiDAR through a transformer and evaluates the result in closed-loop CARLA, a different evaluation setting from the open-loop one this paper uses. AutoE2E makes a narrower commitment: it does not reproduce the detection/tracking/mapping/occupancy chain, only the idea of a shared BEV representation feeding a planner, plus explicit navigation context and auxiliary route reconstruction. It has not been compared against UniAD on a common benchmark, and no claim of superiority or inferiority is made.

Temporal BEV representations. How a temporal fusion strategy folds history into the current frame determines what the resulting representation can be attributed to: a given moment, a window, or an unbounded past. BEVFormer [4] aggregates multi-camera features into a BEV representation spatiotemporally, using learnable BEV queries and deformable attention derived from Deformable DETR [6]. BEVFormer v2 [5] adapts modern image backbones to BEV perception through perspective supervision, and releases an official ResNet-50 checkpoint together with a temporal configuration that fuses multi-frame BEV by convolution – an explicit, bounded window rather than a recurrent state carried indefinitely. AutoE2E's frozen camera pathway starts from this official checkpoint (Sec. 2).

Map- and route-conditioned planning. ChauffeurNet [14] rendered the road map and the intended route as rasters for imitation learning; VectorNet [15] and LaneGCN [16] encoded HD maps as vectors and lane graphs, showing that map structure governs motion-forecasting accuracy. The UniAD-family baselines evaluated on the KITScenes [12] E2E benchmark are conditioned on discrete navigation commands (turn left, turn right, go straight); the route used here is instead a lane-sequence corridor over a Lanelet2 [13] map with a destination marker, fitted *after the fact* to the trajectory the model is then scored against (Sec. 2). That provenance is precisely why route conditioning cannot be credited with an accuracy gain from this protocol alone; isolating its effect requires a same-scene experiment not yet run (Sec. 6).

Evaluation protocols and benchmarks. Open-loop planning evaluation on nuScenes has repeatedly been shown to admit shortcuts by which high scores are obtained without genuine driving competence [17, 18].

Closed-loop evaluation in nuPlan [10], non-reactive simulation in NAVSIM [20], the finding that rule-based planners can rival learned ones under the standard protocol [19], and conditioning-signal shortcut biases [21] are different instances of the same finding: change what the evaluation condition reveals or withholds, and the ranking of models changes with it, independent of what they actually learned. This paper remains in the open-loop setting and adopts three design principles that follow from this literature: align the evaluation setup with the deployment contract, distinguish deterministic overlay replay (which does not re-run the model) from checkpoint-internal validation (which does), and do not read differences between distinct scene populations as causal effects.

2 Model Architecture

AutoE2E did not start as a map- and route-conditioned model. The starting point was a temporal camera-only BEV planner built on BEVFormer v2 [5]; the working group's contribution on top of it is a navigation pathway that conditions this baseline on an HD map and a route. This section covers both phases, the deployment contract they share, the control planner, and the resulting parameter budget.

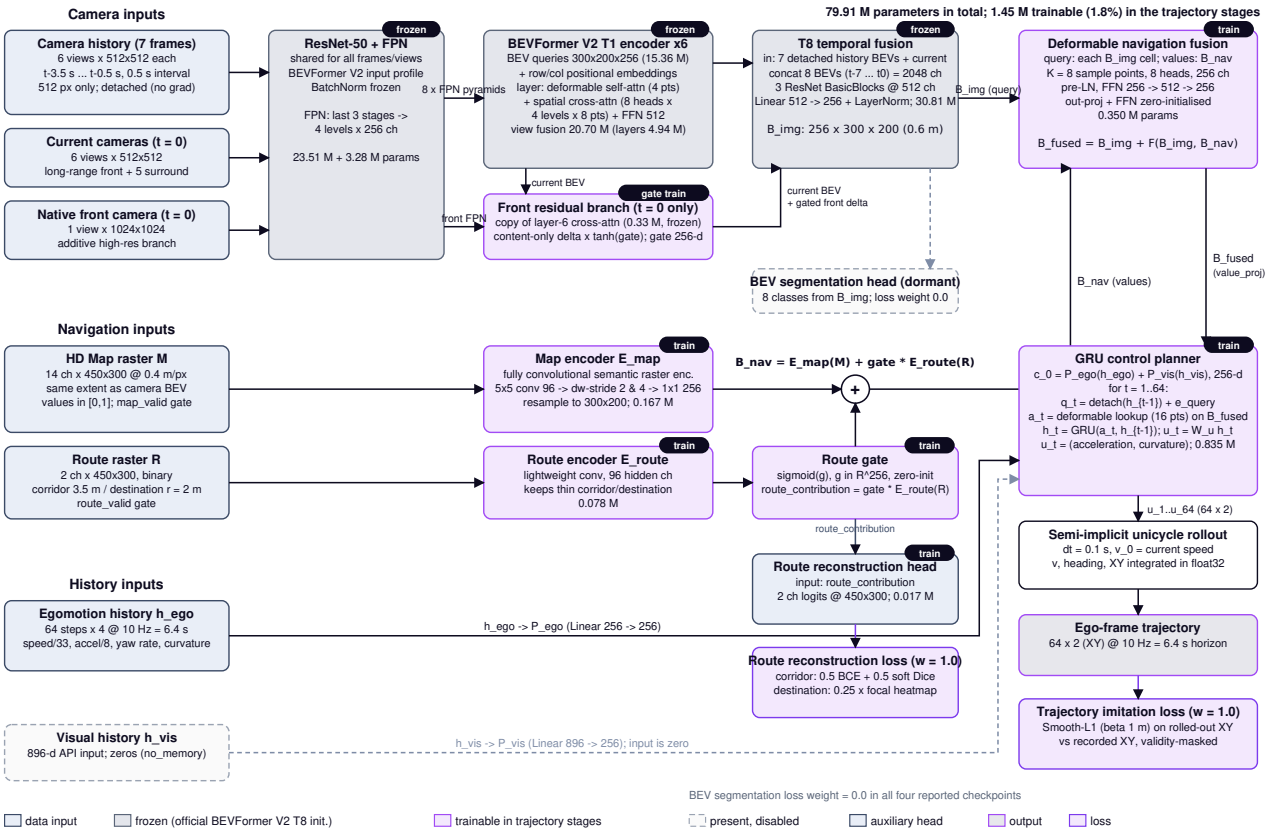


Figure 2: The evaluated map- and route-conditioned AutoE2E configuration. At each of eight times, six 512×512 camera views are processed: the current time plus seven history times. A 1024×1024 front view is added only at the current time. A frozen ResNet-50 + FPN and six-layer single-frame BEVFormer V2 encoder are applied independently per time, followed by T8 convolutional fusion across the eight BEVs; a navigation pathway encodes the 14-channel map and the 2-channel route separately, gated summation, deformable navigation fusion, 6.4 s of ego-motion history, a GRU control planner, and unicycle integration. All reported counts are parameter counts measured by instantiating the implementation (79,906,522 total; 1,448,582 trainable). Dashed boxes denote branches present in the checkpoint but disabled.

2.1 Deployment Contract

At every frame, the deployed AutoE2E policy receives: six-view camera images (the current frame plus 7 history frames at 0.5 s intervals, each 512×512), a 1024×1024 front camera image for the current frame, ego-centered HD-map rasters (14 channels) and a selected-route raster (2 channels), and 6.4 s of ego-motion history at 10 Hz. The output is 6.4 s of acceleration and curvature at 10 Hz. The primary evaluation is conducted under the

condition in which all of these inputs are available; because situations in which the map or the route is missing can occur in operation, behavior from cameras alone is measured as a separate track (Sec. 5).

2.2 Phase 1: Temporal Camera BEV Baseline

The baseline AutoE2E configuration – the architecture the working group started from – is the frozen BEVFormer V2 T8 temporal camera pathway alone, with no HD-map or route input: six camera views per frame, fused temporally across eight timesteps, feeding directly into the control planner (Sec. 2.4).

The camera pathway builds on the official BEVFormer V2 R50 T8 checkpoint [5]. A four-level, 256-channel feature pyramid is formed from the last three stages of ResNet-50 [22], and a six-layer BEVFormer encoder updates 300×200 learned BEV queries (256-dimensional, 15.36 M parameters). The BEV grid covers X from −60 m to 120 m and Y from −60 m to 60 m in ego coordinates, at 0.6 m cells (Fig. 3). Each layer consists of deformable self-attention over two queues obtained by duplicating the current BEV (8 heads, 4 points each), multi-scale spatial cross-attention that maps reference points at four pillar heights ($z \in [-5, 3]$ m) into the images through a calibrated projection operator (8 heads, 4 levels, 8 points each), a 512-dimensional FFN, and three LayerNorms. The projection operator implements pinhole and fisheye (F-theta) models, so this evaluation is not calibration-free.

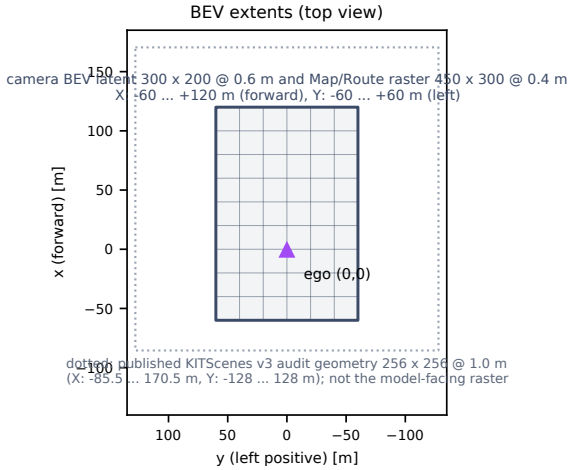


Figure 3: BEV spatial extents. The camera BEV latent (300×200, 0.6 m) and the map and route rasters fed to the model (450×300, 0.4 m) cover the same extent (X: −60 to 120 m, Y: −60 to 60 m). Dotted: the publicly documented KITScenes v3 audit geometry (256×256, 1.0 m; X: −85.5 to 170.5 m, Y: −128 to 128 m) – not the raster the model receives in this evaluation.

In the T8 temporal configuration, seven history frames at 0.5 s intervals ($t-3.5$ s to $t-0.5$ s) and the current frame are passed through the same backbone and encoder, with gradients detached for the history BEVs. The eight BEVs are concatenated into 2048 channels, passed through a three-block ResNet-style fusion module with 512 intermediate channels, and projected to 256 channels to obtain B_{img}:

$$B_{img} = T\left(B_{-7}, B_{-6}, \ldots, B_{-1}, B_0\right). \tag{1}$$

The camera pathway therefore observes 3.5 s at 2 Hz, a temporal span different from the 6.4 s ego-motion history used by the planner (Fig. 4).

The current-frame front camera is additionally processed at 1024×1024: a branch consisting of a frozen copy of the last encoder layer's cross-attention computes a content-dependent delta δ from the high-resolution front features and adds it as $B_0 = B_{\text{enc}} + \tanh(g) \odot \delta$. Since g is 256-dimensional and zero-initialized, the output of the pretrained camera pathway is unchanged at the start of training; these 256 parameters are the only camera-side parameters excluded from freezing.

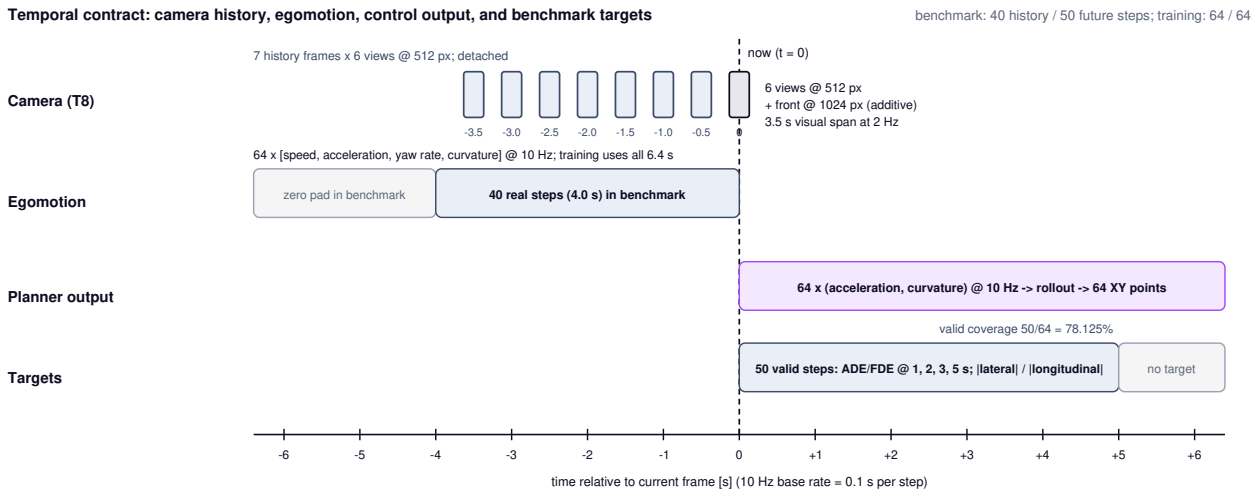


Figure 4: Temporal contract. The camera branch uses 8 frames at 0.5 s intervals (3.5 s), the ego-motion history uses 64 steps at 10 Hz (6.4 s), and the output is 64 steps at 10 Hz (6.4 s). Under the KITScenes benchmark protocol, only 40 history steps (4.0 s, the remainder zero-padded) and 50 future steps (5.0 s) are real data, so the effective horizon coverage of external replay is 50/64 = 78.125%.

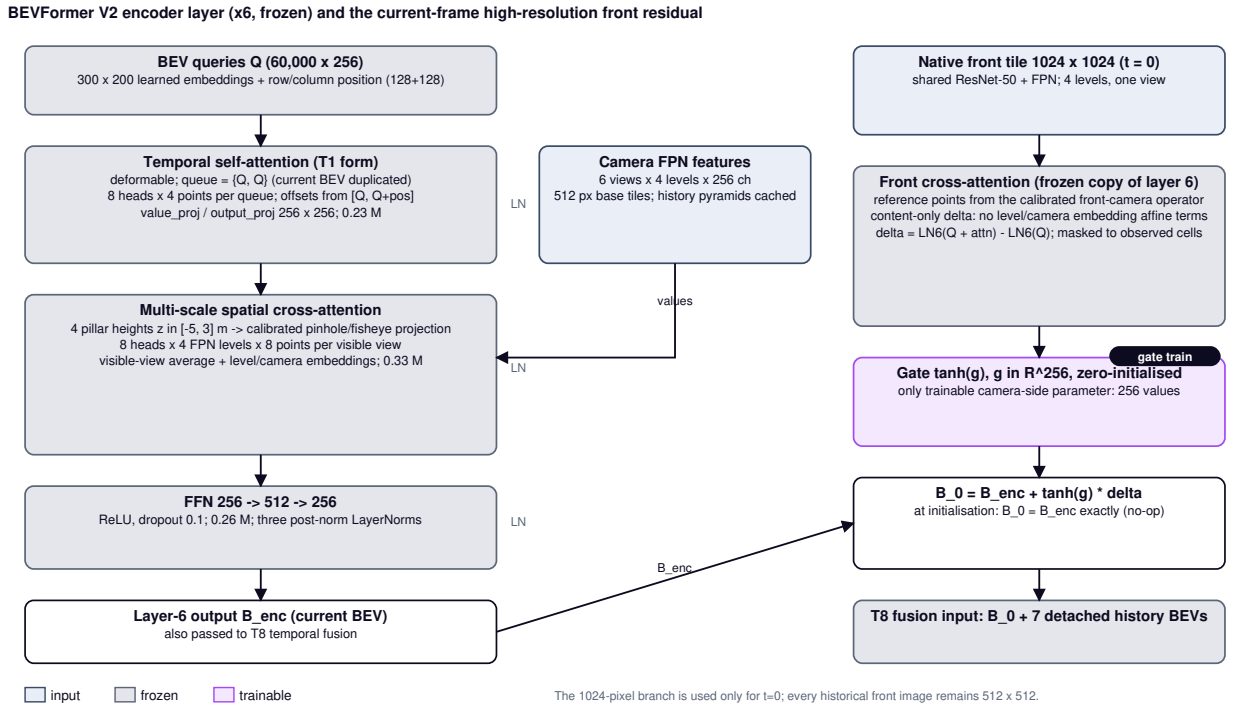


Figure 5: BEVFormer V2 encoder layer (left, six layers, frozen) and the front-camera residual branch (right). The front branch is a frozen copy of the last layer's cross-attention that computes only a content-dependent delta from the 1024×1024 current front image, multiplies it by the tanh of a zero-initialized 256-dimensional gate, and adds it to the current BEV – exactly the identity at initialization. This gate is the only camera-side parameter that remains trainable after freezing; history frames use 512×512 only.

2.3 Phase 2: Map- and Route-Conditioned Extensions

The working group's contribution on top of the baseline above is a navigation pathway that encodes an HD-map raster and a route raster separately, and fuses them into the camera BEV through a gated residual and deformable cross-attention. This is the current working version of AutoE2E, and the one all results in this paper are reported for.

The HD map and the route are represented as ego-centric semantic rasters, 450×300 cells at 0.4 m/px, covering the same extent as the camera BEV. Table 1 defines the 14 map-raster and 2 route-raster channels; Fig. 6 renders them for a real KITScenes validation sample. Binary channels store 1, direction channels store

$(\sin\theta + 1)/2$ and $(\cos\theta + 1)/2$, and road level stores $(\text{clip}(\text{level}, -8, 8) + 8)/16$, so all values lie in $[0, 1]$. The route corridor is drawn from the sequence of lane centerlines with a width of 3.5 m (truncated 10 m behind the ego vehicle), and the destination marker is a circle of radius 2.0 m. Map validity and route validity are explicit per-sample gates, and invalid rasters are multiplied by zero.

Table 1: Definitions of the 14 map-raster and 2 route-raster channels.

ch	Name	Source primitive (Lanelet2 [13])	Value	Mean occ. [%]
0	Drivable area	Lanelet polygons (excluding crosswalks)	binary	15.55
1	Lane boundary	Left/right boundary lines (width 1.0 m)	binary	8.29
2	Lane centerline	Centerline (width 1.0 m)	binary	5.52
3	Intersection	Lanelet polygons with a turn_direction attribute	binary	1.46
4	Crosswalk	Polygons of subtype crosswalk	binary	0.96
5	Stop line	Stop-line polylines (width 1.0 m)	binary	0.18
6	Static traffic light	`traffic_light` regulatory elements (radius 1.0 m)	binary	0.07
7–8	Heading sin/cos	Centerline orientation (width 3.5 m)	(sin, cos)	14.72
9	Heading valid	Same as above	binary	14.72
10	Known map area	Map boundary polygon	binary	96.98
11–13	Road level, valid, ambiguity	`layer`/`level` attributes	mixed	0.00
R0	Selected route corridor	Centerlines of the route lane sequence (width 3.5 m)	binary	1.45
R1	Destination marker	Route endpoint (radius 2.0 m)	binary	0.05

Fig. 7 shows how these rasters are produced upstream of the model. The KITScenes route is the lane sequence obtained by fitting the ego trajectory over the whole scene to the Lanelet2 map, and the destination is the ego position at the end of the scene. The future ego points themselves are not drawn, but the route is a retrospectively determined sequence of already-driven lanes – that is, it equals the intent the driver actually followed. Because this fact bears directly on how route conditioning should be interpreted, we return to it in Sec. 6.

The static map and the selected route have separate encoders (Fig. 8). The map encoder E_{map} is a fully convolutional network in which a 5×5 convolutional stem (96 channels) and a context path reduced to strides 2 and 4 by depthwise convolutions (192 channels) are aligned to 256 channels by 1×1 projections and added, with the input bilinearly resampled to 300×200. The route encoder E_{route} is a lightweight convolutional network with 96 hidden channels, designed so the thin route corridor and destination features are not lost to coarse pooling. The route contribution is controlled by a 256-dimensional per-channel gate and added to the encoded map:

$$B_{nav} = E_{map}(M) + \sigma(g) \odot E_{route}(R). \tag{2}$$

The gate g is zero-initialized, so its weight at the start of training is 0.5.

The navigation BEV is fused into the camera BEV by deformable cross-attention [6]:

$$B_{fused} = B_{img} + F_{deform}\left(B_{img}, B_{nav}\right). \tag{3}$$

Each image-BEV cell acts as a query, predicts 8 offsets from its own position as the reference point, bilinearly samples B_{nav}, and aggregates the samples with 8-head softmax weights – reducing what would be $O(N^2)$ for dense cross-attention over 60,000 cells to $O(NK)$. Because the output projection and the final FFN linear layer are zero-initialized, the fusion does not alter the representation of the pretrained camera pathway at the start of training, acting as a stabilization mechanism when attaching new navigation features to a frozen visual BEV representation.

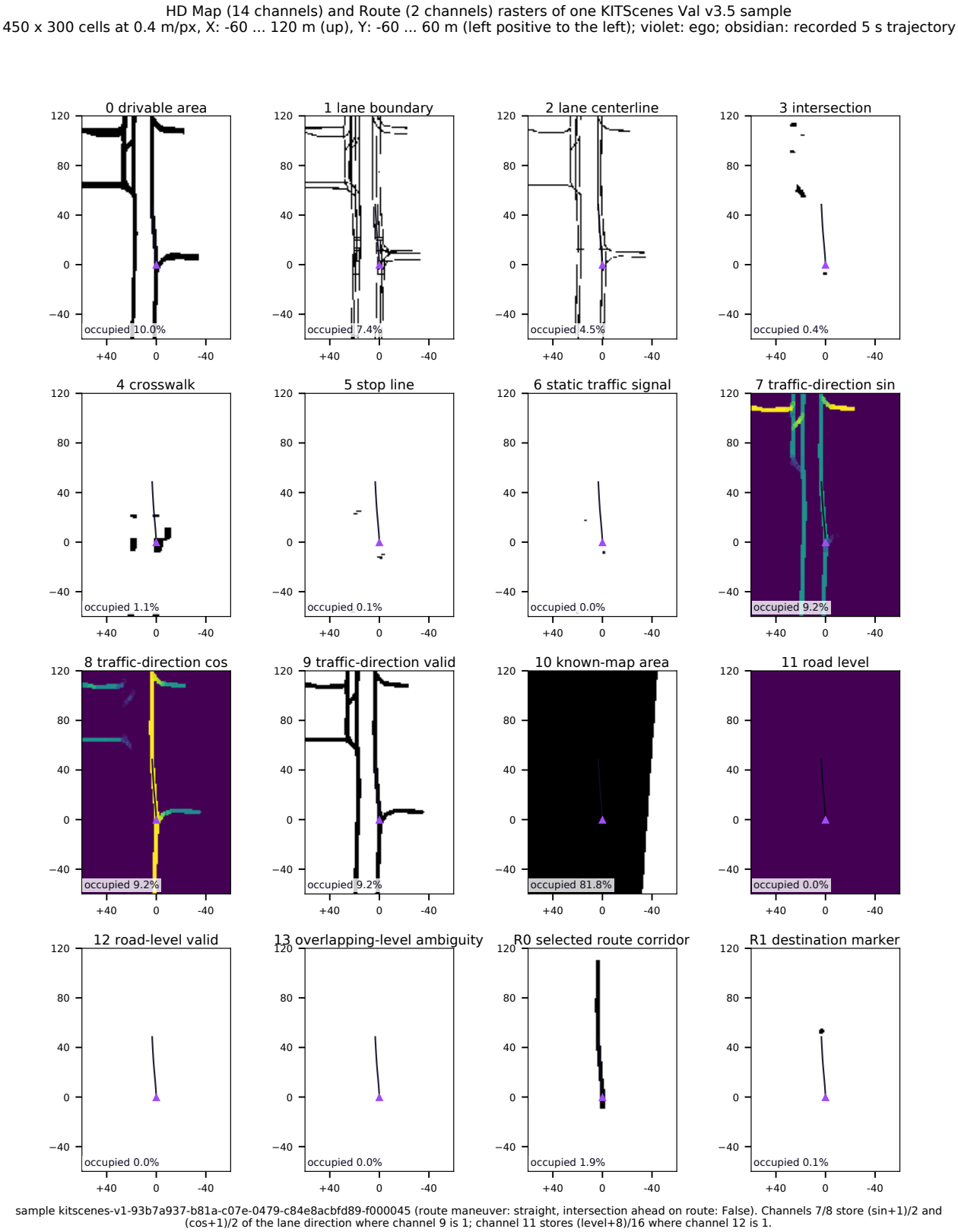


Figure 6: The 14 map channels and 2 route channels for one KITScenes validation sample. These are real data taken from the working group's public Hugging Face dataset viewer for this mirror; violet marks the ego position, obsidian the recorded 5 s trajectory. Road-level channels 11–13 were empty in this sample.

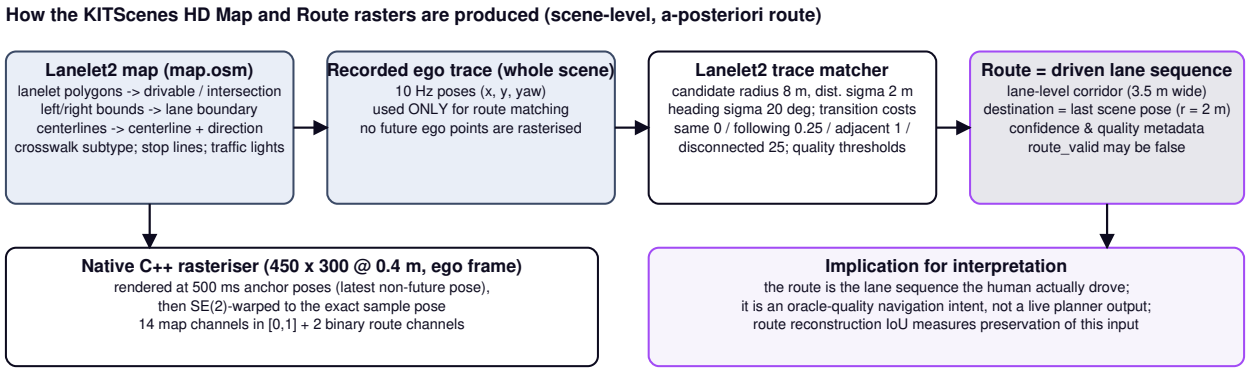


Figure 7: Generation of the map and route rasters in KITScenes. Primitives are extracted from the Lanelet2 map, and the ego trajectory over the whole scene is fitted to a lane sequence using HMM-like transition costs to yield the route. The route is not the output of a live route planner but a retrospective lane-level driving intent.

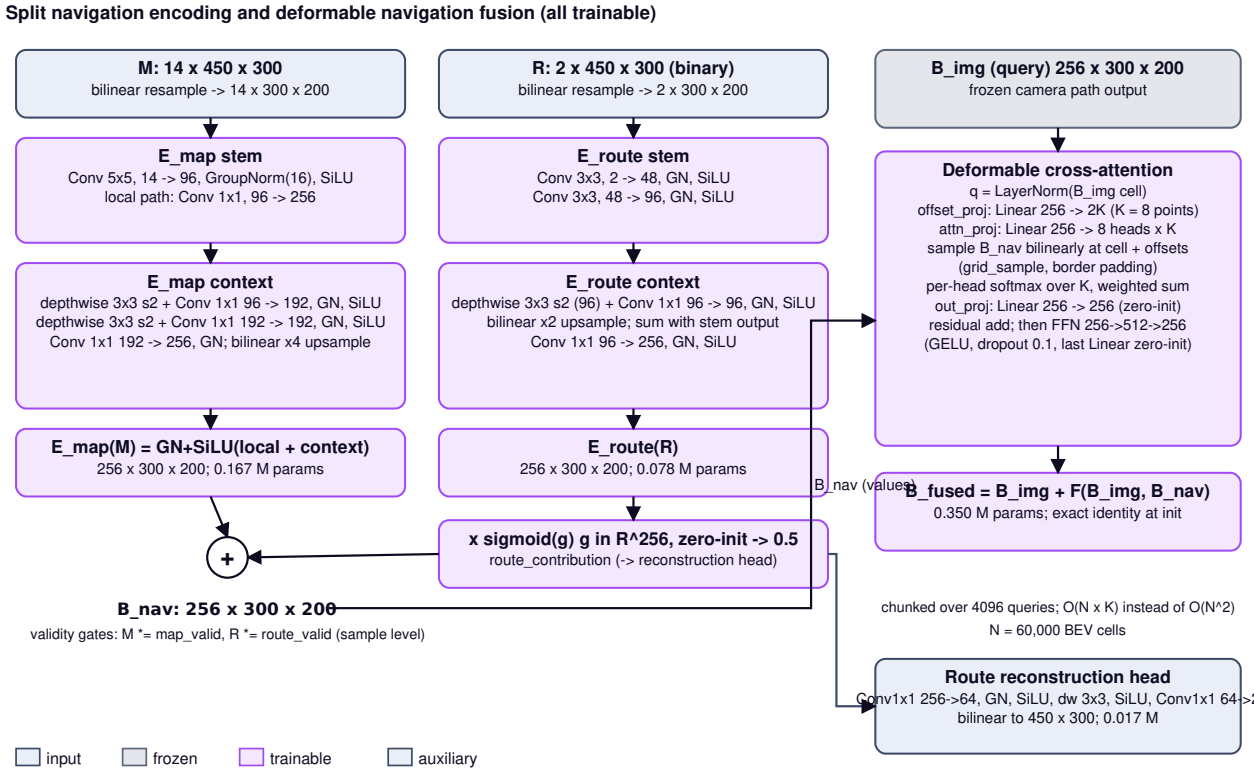


Figure 8: Internal structure of the separate navigation encoding and the deformable navigation fusion. Left: map encoder E_{map} (0.167 M) and route encoder E_{route} (0.078 M), the per-channel gate, and the addition. Right: deformable cross-attention that uses each cell of B_{img} as a query and samples 8 points on B_{nav} (8 heads, 256 channels, 0.350 M). The output projection and the final FFN linear layer are zero-initialized, so the fusion is exactly the identity at the start of training.

2.4 Control Planner

Both phases feed the same fused BEV into one control planner. The ego-motion history h_{ego} is the flattened 256-dimensional vector of [speed, acceleration, yaw rate, curvature] over 64 steps at 10 Hz. The model API retains an 896-dimensional visual-history input h_{vis}, reserved for the future world-action band (Sec. 7); the evaluated checkpoints feed it as zero. The planner is a deterministic GRU (Fig. 9):

$$c_0 = P_{ego}(h_{ego}) + P_{vis}(h_{vis}), \qquad a_t = A_{deform}\left(q_t, B_{fused}\right),\ q_t = \mathrm{sg}(h_{t-1}) + e, \tag{4}$$

$$h_t = \mathrm{GRU}\left(a_t, h_{t-1}\right), \qquad u_t = W_u h_t = \left(a_t^{acc}, \kappa_t\right). \tag{5}$$

At each of 64 future steps, 16 points on the fused BEV are attended to deformably and the GRU updates its state and emits acceleration and curvature. A semi-implicit unicycle model then integrates the control sequence into ego-frame XY ($\Delta t = 0.1$ s):

$$v_t = \max\left(v_{t-1} + a_t^{acc}\Delta t,\, 0\right), \quad \theta_t = \sum_{i \le t} v_i \kappa_i \Delta t, \quad x_t = \sum_{i \le t} v_i \cos\theta_i \Delta t, \quad y_t = \sum_{i \le t} v_i \sin\theta_i \Delta t. \tag{6}$$

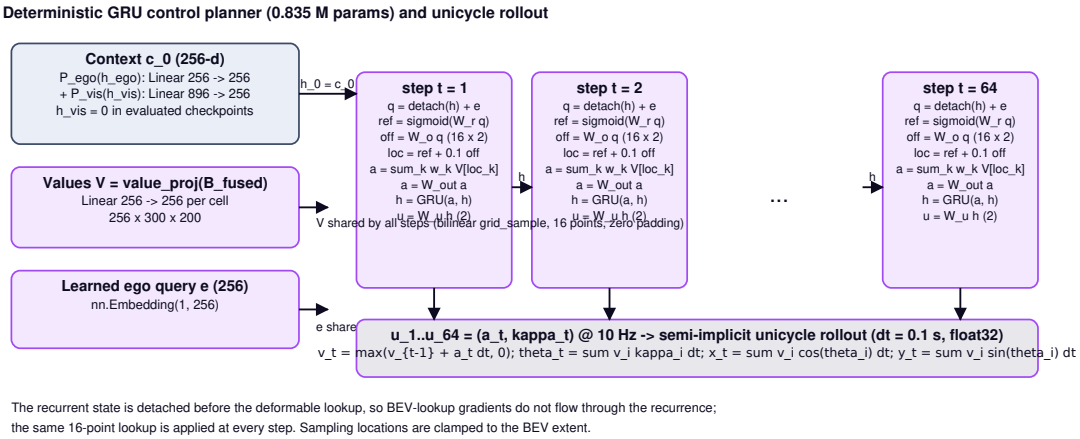


Figure 9: The deterministic GRU control planner (0.835 M parameters). The Bezier and flow-matching planner heads present in the codebase are not used by the evaluated checkpoints.

The route-reconstruction head decodes two channels of logits at 450×300 from the gated route contribution, supervised by the mean of BCE and soft Dice for the corridor and a heatmap focal loss (weight 0.25) for the destination. The trajectory loss is Smooth-L1 ($\beta = 1$ m) between the integrated XY and the recorded trajectory. All reported checkpoints were trained with a trajectory weight of 1.0, a route-reconstruction weight of 1.0, and a BEV-segmentation weight of 0.0 – the segmentation head exists in the checkpoint but is not jointly optimized in this release.

2.5 Model Size

Table 2 and Fig. 10 report parameter counts obtained by instantiating the model in the evaluated six-view configuration. In total-capacity terms, the model is almost entirely a pretrained, frozen visual representation: 97.98% of all parameters (78,292,940 of 79,906,522) belong to the frozen camera-BEV pathway, split mainly between T8 temporal fusion, ResNet-50, and the learned BEV queries. The remaining 1,448,582 parameters (1.81%) are updated during trajectory training, allocated across the GRU planner, the deformable navigation fusion, the map/route encoders, and the route-reconstruction head. At 79.9 M total parameters, AutoE2E is roughly two orders of magnitude smaller than the ∼4 B-parameter and ∼10 B-parameter vision-language-action planners it is benchmarked against (Sec. 5) – a deliberately light footprint for real-time, on-vehicle inference rather than research-scale capability.

Table 2: Parameter counts for the evaluated six-view configuration.

Component	Parameters	State
ResNet-50 backbone	23,508,032	frozen
Feature pyramid (4 levels)	3,278,592	frozen
BEV queries 300×200×256	15,360,000	frozen
Row, column, level, and camera embeddings	66,560	frozen
Encoder, 6 layers	4,940,928	frozen

Table 2: Parameter counts for the evaluated six-view configuration.

Component	Parameters	State
Front cross-attention (copy)	328,960	frozen
Front residual gate	256	trained
T8 temporal fusion	30,809,856	frozen
Map encoder E_{map}	167,200	trained
Route encoder E_{route}	77,984	trained
Route gate	256	trained
Deformable navigation fusion	350,288	trained
GRU control planner	835,380	trained
Route reconstruction head	17,218	trained
BEV segmentation head (weight 0.0)	165,000	frozen
Total / trainable	79,906,522 / 1,448,582	—

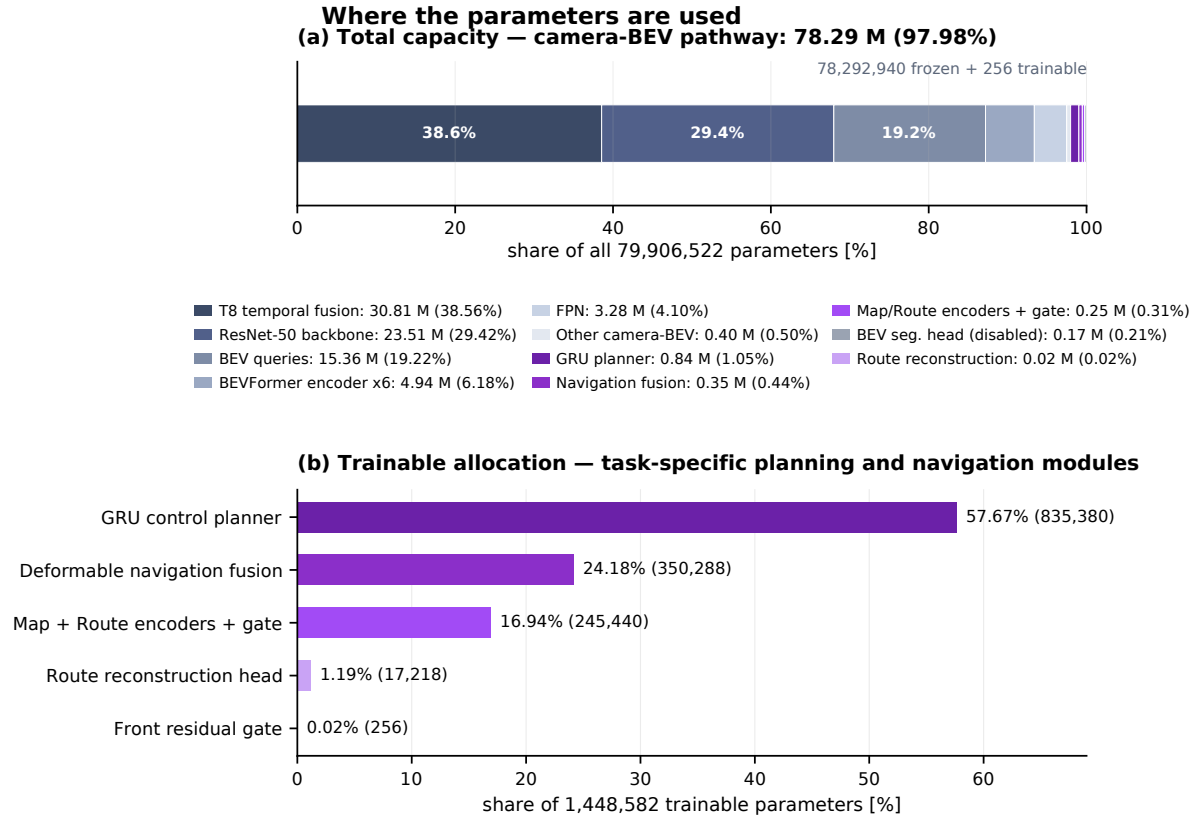


Figure 10: Parameter allocation. (a) The frozen camera-BEV pathway is 97.98% of all parameters. (b) Among the 1.45 M trainable parameters, the GRU planner is 57.7%, deformable navigation fusion 24.2%, and the map/route encoders plus route gate 16.9%.

3 Training

3.1 Datasets

Two datasets anchor training and evaluation. **nuPlan** [10], a large-scale, multi-city closed-loop planning benchmark, is used for the initial trajectory-and-route training stage with the camera BEV backbone frozen at its official pretrained weights. **KITScenes** [12], a multimodal European driving dataset from the Karlsruhe Institute of Technology (KIT) pairing high-fidelity camera and sensor data with complete Lanelet2 [13] HD maps, is used to fine-tune the nuPlan-trained checkpoint to the Karlsruhe road network and camera configuration, and for the primary external evaluation (Sec. 5). Both datasets carry their own licenses, which the released checkpoint weights inherit; the model architecture, training code, and evaluation pipeline are Apache-2.0 regardless of which dataset trained a given checkpoint (Sec. 10).

The split manifest identifies the KITScenes fine-tuning source as the official training split: 129 empty scenes are excluded from 533 available train scenes, leaving 404 eligible scenes and 42,667 samples. A scene-level 90%/10% split yields 38,847 samples from 364 scenes for training and 3,820 samples from 40 scenes for frozen

internal validation. Because these 404 scenes belong to the official training split, their scene IDs are disjoint from the official 117-scene Val and 206-scene Test populations used for evaluation.

3.2 How It Was Trained

Training proceeds in two stages (Fig. 11). The **baseline** stage trains the trajectory and route pathways on nuPlan with the camera BEV frozen at its official pretrained weights: 8 distributed GPU workers, seed 149, learning rate `1e-4`, trajectory weight 1.0, route-reconstruction weight 1.0, BEV-segmentation weight 0.0, and 1,024 internal validation samples. The resulting checkpoint is released as `nuplan-epoch-5`. The **fine-tuning** stage then freezes that checkpoint as parent and continues training on KITScenes at learning rate `3e-5`, with the same 8-worker, seed-149, loss-weight configuration, over the 38,847-sample / 364-scene training split described above, validated internally on 3,820 samples / 40 scenes in BF16 precision. The result is released as `kitscenes-epoch-5`, the checkpoint fine-tuned for the Karlsruhe road network and camera configuration, and the one reported as the primary result throughout Sec. 5. An optional BEV-only training stage (unfrozen camera BEV, BEV-segmentation loss) exists in the code path; its use as parent of the released baseline checkpoint is not recorded in the available metadata. Neither released checkpoint passed the repository's automated checkpoint quality gate.

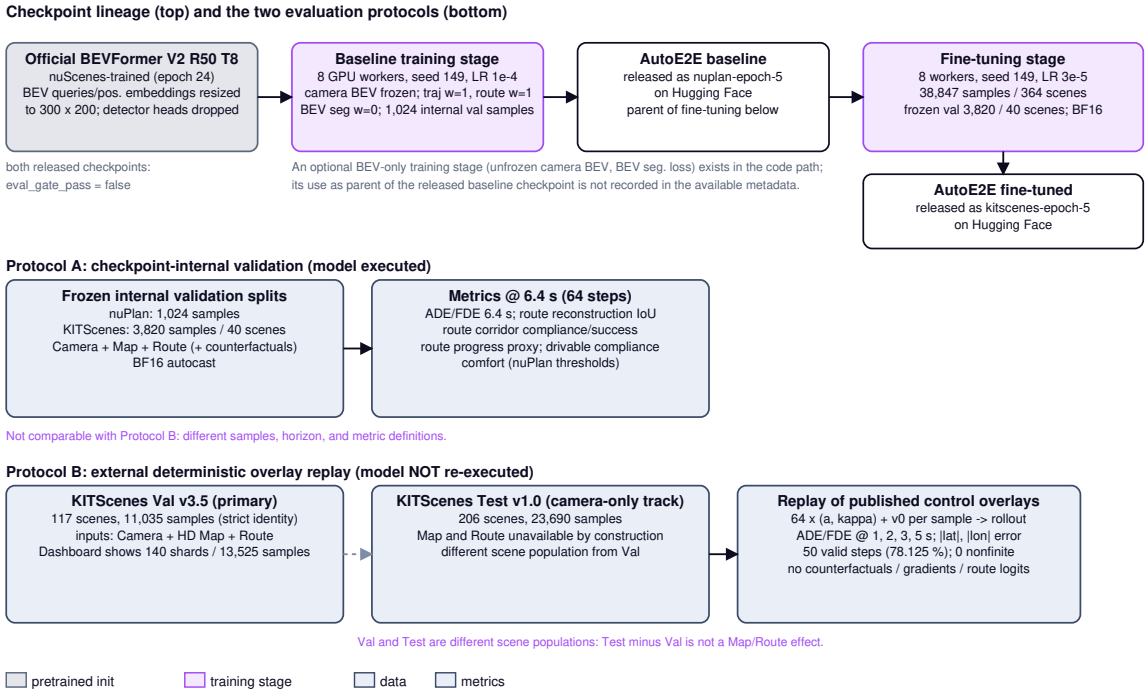


Figure 11: Checkpoint lineage (top) and the two evaluation protocols (bottom). Protocol A is checkpoint-internal validation that runs the model; Protocol B is replay of the released deterministic control overlays. The two are never mixed in a single ranking table. Val and Test are different scene populations.

3.3 Where It Was Trained

Training runs on an EKS Auto Mode Kubernetes cluster on AWS, chosen over a standard EKS node group plus Karpenter setup for its built-in CNI, autoscaler, and load-balancer controller, which removes most of the day-two operational overhead of running GPU infrastructure. Imitation learning uses Ray Train jobs admitted by Kueue's GPU-quota scheduler onto `p5en.48xlarge` Capacity Block reservations (4- or 8-GPU jobs, whole-GPU allocation to preserve intra-instance communication), with a smaller retained `g6.2xlarge` pool reserved for inference, validation, and canary jobs. Closed-loop simulation was evaluated and deliberately not adopted for this release: CARLA's Vulkan dependency is incompatible with the Bottlerocket AMI that EKS Auto Mode's GPU nodes run, so the working group's closed-loop path instead goes through offline reinforcement learning (Implicit Q-Learning) on the same WebDataset shards used for imitation learning, requiring no simulator. The likely launch configuration for training the released checkpoints, traced from the training branch, is a single `p5en.48xlarge` (8×H200) or `p5.48xlarge` (8×H100) Capacity Block, 8-way distributed, consistent with the epoch, learning-rate, and seed values above; this is an inferred hardware configuration, not a recorded one, and total training wall-clock time and GPU-hours are not available in the evidence used for this paper. Every

stage – data ingest, imitation-learning training, open-loop evaluation gating, and offline-RL refinement – logs experiments, metrics, and artifacts to MLflow, orchestrated as Flyte workflows; both are deployed on the same cluster and reachable through internal, Cognito-authenticated CloudFront endpoints rather than the public internet.

4 The DataModelConsole Deliverable

Alongside the model weights and training code, this release ships **DataModelConsole**, live at `https://d2itskdqq39tx1.cloudfront.net/`, a read-only dashboard that replays synchronized camera scenes with ground-truth and predicted trajectories in both camera and bird's-eye views, next to dataset versions, MLflow experiment runs, and Flyte pipeline executions. It is the artifact behind both qualitative results in this paper: the KITScenes Val/Test benchmark tables of Sec. 5 are scored from the same dataset shards and control-overlay blobs the dashboard serves, and the qualitative behavior captures of Sec. 5.6 (Fig. 16) are curated recordings made with the dashboard on real-world driving beyond the KITScenes cities. The dashboard is implemented as a Go API over MLflow/Flyte/S3 state and a Next.js web frontend, and covers dataset and shard browsing, per-sample trajectory overlays, an occupancy view, reasoning-label review (for the future reasoning band), and experiment/run browsing – the working group's own tool for auditing what a checkpoint actually predicts on a given scene before trusting a benchmark number. It is released alongside the code at the tag in the header of this paper, and is intended as public-facing infrastructure the broader community – including the institutions in Sec. 8 – can use to independently inspect AutoE2E's behavior rather than take its benchmark table on faith.

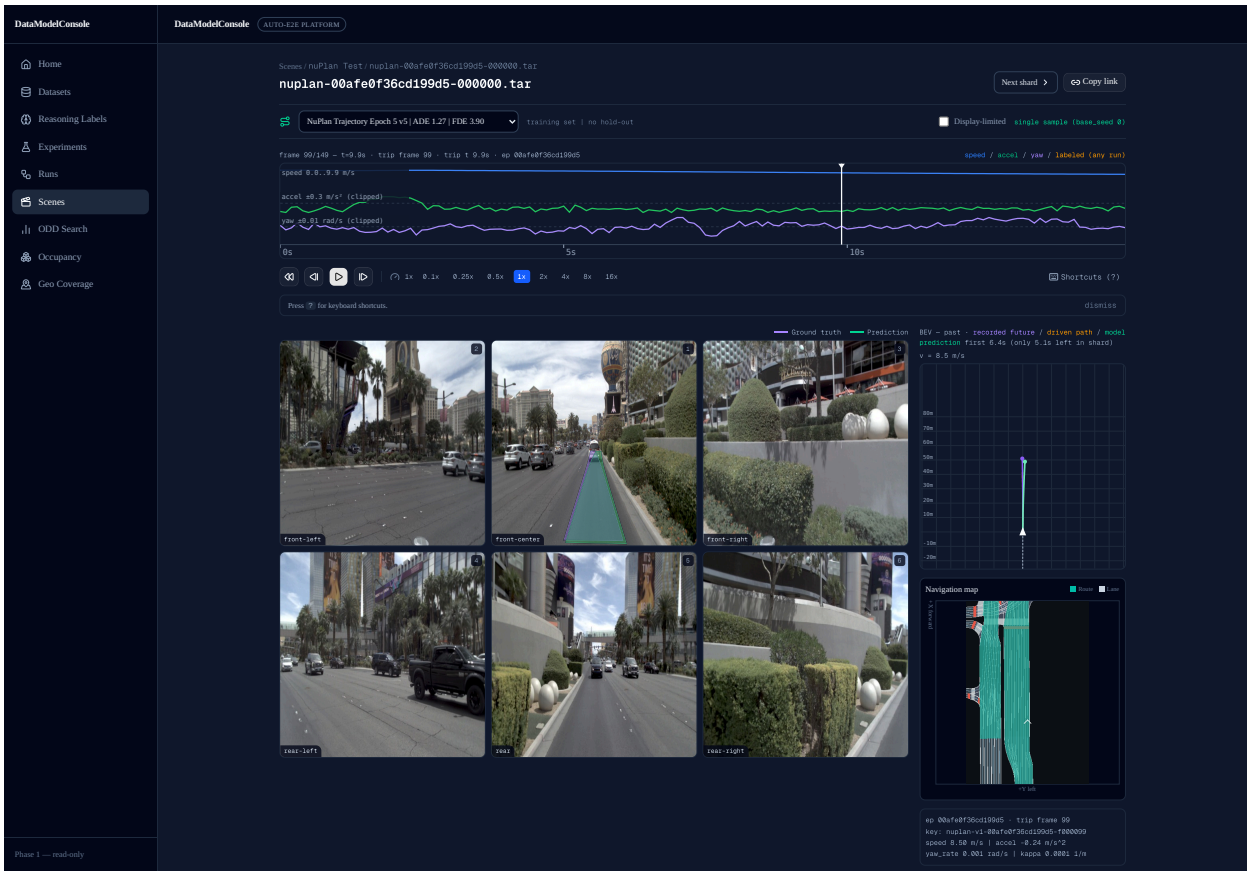


Figure 12: The DataModelConsole dashboard: the six-camera mosaic, a speed/acceleration/yaw timeline scrubber, the bird's-eye-view panel (ground truth vs. prediction), and the navigation map, alongside the sidebar's dataset, experiment, run, and occupancy browsers.

5 Benchmarks

5.1 Population and Protocol

The primary benchmark is the KITScenes validation split, where Camera + HD Map + Route are all available, computed over the 117 scenes and 11,035 samples that satisfy strict identity with the official Map and Route evaluation. Because the KITScenes test split does not release maps, it is evaluated as a camera-only robustness and transfer track over all 206 scenes and 23,690 samples. Since Val and Test are different scene populations, the Test-versus-Val difference is not interpreted as a causal effect of the map and route. The external evaluation is a replay of the published deterministic control overlays; it does not re-run the model and contains no route counterfactual inputs, input gradients, or route-reconstruction outputs. Checkpoint internal validation (Protocol A, Fig. 11) is a separate protocol and is not mixed into the same ranking table as the external replay (Protocol B).

5.2 Models Used for Comparison

Table 3 positions AutoE2E against the systems introduced in Sec. 1.1 on a single scope comparison – what each system takes as input and how it is evaluated, not relative performance. Beyond AutoE2E's own two checkpoints, this paper compares against six other driving policies evaluated on KITScenes-derived populations: **Qwen-Drive-1.0** [1], a vision-language-action model unifying 3D perception, driving VQA, and motion planning, reported at approximately 4 B parameters; **Alpamayo 1.5** [2], NVIDIA's reasoning-focused distillation-teacher model, reported at approximately 10 B parameters; **SimForge**, a multi-camera trajectory model evaluated with 7 camera slots including two virtual telephoto views; **METEOR**, an internal model without a public paper; **VAD** [7], the vectorized-scene planning-oriented baseline, evaluated here with an oracle-conditioned navigation command rather than a realistic one; and **Drive-JEPA**, a video-latent (not BEV) model whose own published benchmarks are NAVSIM and Bench2Drive rather than KITScenes or nuPlan, so its numbers here come from the same cross-model protocol as the others rather than from its original paper.

Table 3: Positioning relative to representative E2E driving systems. This table delineates the scope of the claims made here and does not indicate relative performance.

System	Camera-only prim. eval.	HD map input	Route input	Temp. BEV	Unified det./track./occ.	Closed-loop eval.	Evaluation data
UniAD [3]	○	— (online map est.)	Navigation command	○	○	—	nuScenes
VAD [7]	○	— (vector map est.)	Navigation command	○	Partial	○ (CARLA)	nuScenes / CARLA
TransFuser [8]	— (camera + LiDAR)	—	Target point	—	Aux. tasks	○ (CARLA)	CARLA
Qwen-Drive-1.0 [1]	○	— (online est.)	n/a	—	○ (3D perc.)	—	nuScenes
Alpamayo 1.5 [2]	— (camera + nav.)	n/a	○ (nav. command)	n/a	—	○ (AlpaSim)	PhysicalAI-AV, AlpaSim
METEOR	n/a	n/a	n/a	n/a	n/a	n/a	internal, unpublished
Drive-JEPA	○	n/a	n/a	— (video latent)	—	○ (Bench2Drive)	NAVSIM / Bench2Drive
AutoE2E (this paper)	— (secondary track)	○ (14-ch raster)	○ (2-ch raster)	○ (T8, 3.5 s)	— (BEV seg. disabled)	—	KITScenes Val/Test, nuPlan

5.3 Primary Evaluation: KITScenes Val

Table 4 and Fig. 13 report ADE/FDE for AutoE2E's two released checkpoints – fine-tuned and baseline – on 11,035 samples. The fine-tuned checkpoint is lowest at every horizon (5 s: ADE 1.94 m, FDE 5.56 m) and is the checkpoint submitted for official KITScenes leaderboard evaluation; the baseline is inferior to it throughout, at 5 s FDE 6.39 m.

Table 4: KITScenes Val, Camera + HD Map + Route, external overlay replay (n = 11,035 samples per model). Units m; lower is better.

Model	ADE 1 s	ADE 2 s	ADE 3 s	ADE 5 s	FDE 1 s	FDE 2 s	FDE 3 s	FDE 5 s
AutoE2E (fine-tuned)	0.1472	0.3729	0.7449	1.9405	0.2850	0.9239	2.0196	5.5645
AutoE2E (baseline)	0.1951	0.4723	0.9147	2.2951	0.3779	1.1331	2.4226	6.3885

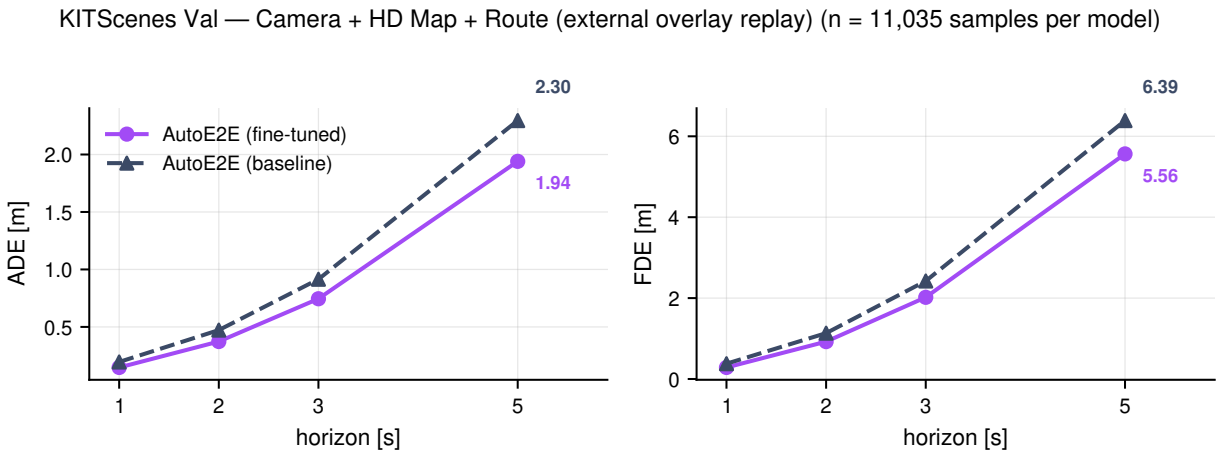


Figure 13: ADE (left) and FDE (right) across horizons on KITScenes Val (Camera + HD Map + Route; n = 11,035). Solid is the fine-tuned checkpoint, dashed the baseline. 5 s values are annotated.

5.4 Secondary Evaluation: KITScenes Test (Camera Only)

Table 5 and Fig. 14 report results on the camera-only track (23,690 samples) for the same two checkpoints. The fine-tuned checkpoint has the lowest reported means at 1 s and 5 s, whereas the baseline is lowest at 2 s and 3 s. As throughout, Val and Test are different scene populations, so this table is not a map/route ablation against Table 4. The fine-tuned checkpoint was trained under a condition in which the map and the route are provided, so it is conceivable that it becomes relatively brittle when they are absent; because Test and Val contain different scenes, geographies, and motion distributions, this gap cannot be quantified as the effect of missing map and route inputs.

Table 5: KITScenes Test, Camera only (map and route unavailable by construction), external overlay replay (n = 23,690 samples per model).

Model	ADE 1 s	ADE 2 s	ADE 3 s	ADE 5 s	FDE 1 s	FDE 2 s	FDE 3 s	FDE 5 s
AutoE2E (fine-tuned)	0.1463	0.4121	0.8263	2.1042	0.3107	1.0453	2.2199	5.9422
AutoE2E (baseline)	0.1602	0.3979	0.7924	2.1120	0.3134	0.9727	2.1621	6.1751

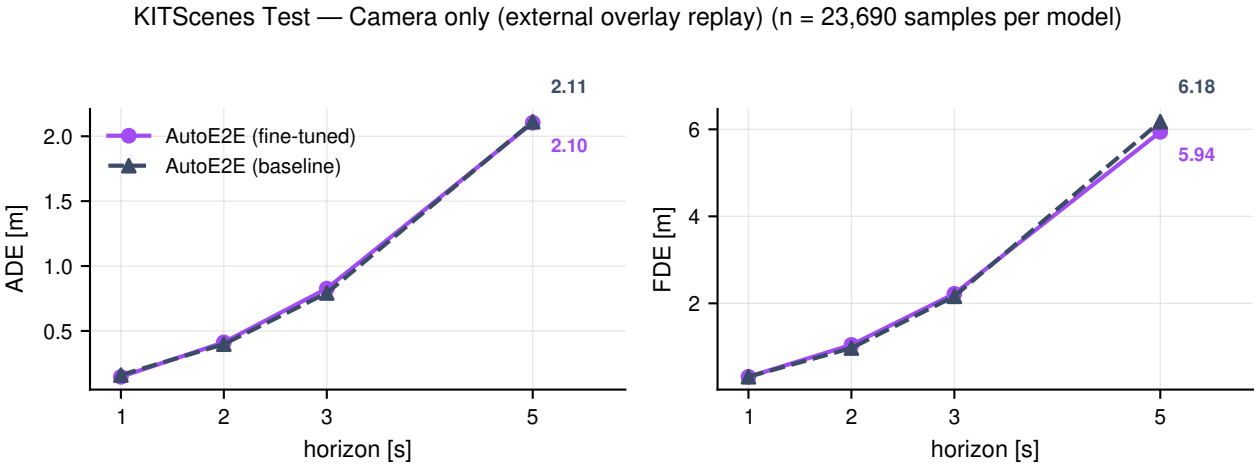


Figure 14: ADE and FDE on KITScenes Test (Camera only; n = 23,690). This is a different scene population from Val, so differences with respect to Fig. 13 do not imply a causal effect of map and route.

5.5 Cross-Model Benchmark

Table 6 and Fig. 15 compare AutoE2E against the six other driving policies of Sec. 5.2. This comparison mixes two populations: five models are scored on a non-overlapping population of 202 anchor keyframes built from the same 117 Val episodes, while AutoE2E's baseline checkpoint, VAD, and Drive-JEPA are scored on the full 11,035-sample Val population. Camera count and preprocessing also differ by model – AutoE2E uses six views at a fixed square resolution, Qwen-Drive-1.0 used three front cameras, Alpamayo 1.5 used four, and METEOR a single frame with no map or route input at all – so this table compares systems under their own native contract, not a controlled ablation of any single input. Among all eight entries, AutoE2E's fine-tuned checkpoint reports the lowest ADE at 1 s, 2 s, and 3 s, and the lowest FDE at 1 s and 2 s; at 3 s and 5 s, Qwen-Drive-1.0 reports the lowest ADE and FDE, with AutoE2E's fine-tuned checkpoint second – at roughly two orders of magnitude fewer parameters.

Table 6: Open-loop ADE/FDE (m) across eight driving policies at the 3 s horizon. Evaluated-on column gives the population: "202 kf" is the 202-anchor non-overlapping keyframe population, "11,035" is the full Val population of Table 4.

Model	Setting	Evaluated on	ADE@3s	FDE@3s
Qwen-Drive-1.0 [1]	first of 6 trajectories	202 kf	0.565	1.397
AutoE2E (fine-tuned)	single	202 kf	**0.744**	**1.980**
SimForge	7 cam. slots, first of 6	202 kf	0.831	2.209
AutoE2E (baseline)	single, zero-shot	11,035	**0.915**	**2.423**
Alpamayo 1.5 [2]	first of 6 trajectories	202 kf	1.020	2.820
VAD [7]	route-guided (oracle command)	11,035	1.242	2.821
METEOR	top-1 of 3 paths	202 kf	1.240	3.282
Drive-JEPA	top-1, perception-based v2	11,035	2.266	4.774

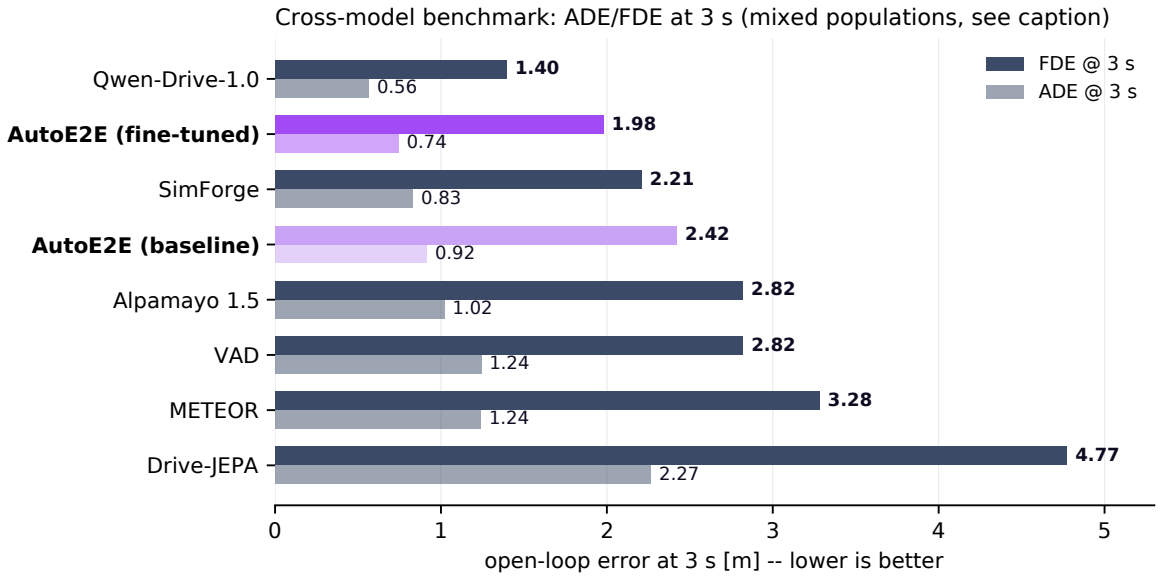


Figure 15: ADE/FDE at the 3 s horizon for the comparison in Table 6.

On the same KITScenes Val scenes, the fine-tuned checkpoint outperformed the zero-shot baseline at every horizon. UniAD and AutoE2E both adopt a BEV-centric representation and planning-oriented feature sharing, but UniAD unifies perception, tracking, mapping, motion forecasting, occupancy, and planning through task interaction, whereas AutoE2E narrows its scope to temporal camera BEV, explicit navigation context, auxiliary route reconstruction, and direct ego planning. No claim of superiority or inferiority relative to UniAD is made, since no common benchmark exists between the two.

The evidence in this paper does not support a causal claim that the HD map and the route improved planning accuracy: Val and Test are different scenes, Test by construction has no map and no route, this report contains no camera-only baseline on the same scenes as Val, and per-scene paired errors have not been computed. What can be said is behavioral: route-corridor compliance (the fraction of steps a 2.0 m footprint remains inside the 3.5 m route corridor) is 0.40–0.42 and success 0.06–0.08 under this strict definition, and a route reconstruction IoU of 0.999 verifies that the route information survives the gate, without being evidence the policy drove along the route.

5.6 Qualitative Examples

Beyond the quantitative KITScenes benchmark above, Fig. 16 shows six real-world behavior captures recorded with the DataModelConsole dashboard (Sec. 4), spanning driving locations outside the KITScenes cities of Sec. 3.1 – evidence that the behaviors below are not an artifact of one city's road layout. Each panel overlays AutoE2E's predicted trajectory (teal) on the camera view and a bird's-eye-view panel against the recorded human driving (violet): slowing for and avoiding a jaywalking pedestrian, giving way to a cyclist cutting in, negotiating around a stopped car, yielding to a pedestrian at a crossing, yielding at a stop sign, and driving in rain and poor visibility. These are curated, single-scene captures selected by the working group to illustrate specific behaviors, not a random or representative sample, and are reported as qualitative illustration alongside, not evidence for, the quantitative claims in Sec. 5.3–5.5.

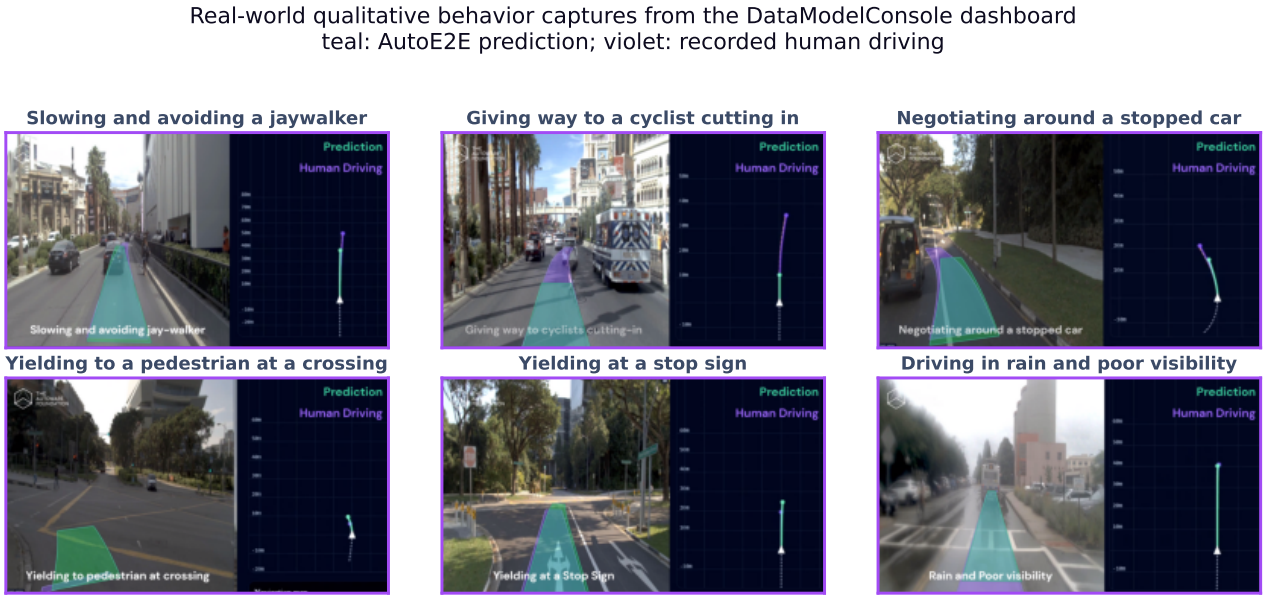


Figure 16: Real-world qualitative behavior captures from the DataModelConsole dashboard (official Autoware Foundation Robotaxi Working Group material). Teal is AutoE2E's predicted trajectory, violet the recorded human driving, overlaid on the front camera view and a bird's-eye-view panel.

5.7 Inference Cost and Real-Vehicle Evidence

Inference reaches up to **76 FPS** (SwinV2-Tiny backbone, feature-concat fusion, RTX 5080, batch 1), and has been benchmarked across consumer and workstation GPUs (RTX 5000 Quadro, A40, RTX 4070/4080/5090). A preliminary real-vehicle result from TU Munich's EDGAR platform – the first operational-hardware evidence of this efficiency advantage – is reported as forward-looking perspective in Sec. 7, rather than as a closed validation here.

6 Limitations

This paper documents an early snapshot of an actively developed reactive band, released at this stage so the working group's evaluation protocol and baseline evidence are open to scrutiny and reuse rather than held back until every gap below is closed; Sec. 7 describes the concrete improvements already planned. AutoE2E is currently an open-loop model: it is not designed as a closed-loop policy, has not been evaluated under closed-loop safety or intervention metrics, and future versions are intended to be trained and evaluated on real vehicles directly rather than assumed safe to deploy on the basis of the open-loop numbers in this paper.

The evaluation in this paper is open-loop only and includes no closed-loop safety or intervention metrics. No controlled map/route experiment on identical scenes was conducted. The external replay is limited to 5 s, so 22% of the 6.4 s output is not evaluated. Val and Test are different populations, and the difference between them carries no causal meaning. The camera encoder is frozen, and the degree to which a representation trained on nuScenes fits the six-camera configuration and 0.6 m grid of KITScenes has not been separately evaluated. The full UniAD task stack is not included. All checkpoint quality gates are recorded as not passed. The training data are nuPlan and KITScenes from a small number of German and multi-city scenes, so distribution shifts in geography, season, and map may exist. Because the route is a post-hoc sequence of already-driven lanes,

robustness to route errors is not measured. Under the KITScenes benchmark protocol, only 4.0 s of the 6.4 s ego-motion history is real data, leaving a distribution gap with respect to training.

Deployment-aligned evaluation should retain the navigation inputs the policy requires, camera-only evaluation should be positioned as a complementary robustness track rather than the primary benchmark, and the real-vehicle and cross-model results of this paper should be read as preliminary evidence pending the full same-scene ablation, not as settled claims.

7 Future Work, Outlook and Collaboration

AutoE2E is the reactive band of a three-band architecture planned for the Autoware Foundation's open-source autonomous-driving stack: a reactive band (this paper), a reasoning band, and a world-action band, decoupled rather than trained as one monolithic network. This paper is explicitly scoped to the frozen-camera-backbone version of AutoE2E described in Sec. 2; unfreezing the camera BEV pathway for joint fine-tuning is left to future work and is not evaluated here. Concretely, the working group's roadmap includes:

- **Unfreezing the camera BEV backbone** for joint fine-tuning with the navigation and control pathways, expected to require its own evaluation protocol rather than a direct comparison with the frozen-backbone results reported here.
- **Replacing the HD-map dependency with a navigation map.** The current deployment contract requires an HD map; a navigation- or SD-map-conditioned variant would relax this requirement toward the maps actually available at scale.
- **Closed-loop training via reinforcement learning** in a photorealistic world model, rather than the open-loop imitation setting evaluated throughout this paper. Early integration work with NVIDIA's AlpaSim closed-loop simulation platform and Gaussian-splatting-based photorealistic rendering is in progress.
- **Activating the reasoning and world-action bands**, Fig. 1's JEPA-style future-feature prediction included, so that the reactive policy evaluated here can be coupled with a higher-level reasoning band and a learned world model rather than being extended into a single larger network.
- **Real-world vehicle deployment**, building on the preliminary TUM EDGAR evidence below toward a driveable, not only demo-capable, system.

8 Collaborations

Technical University of Munich (TUM) – EDGAR Research Platform

A preliminary real-vehicle result, coordinated with the working group by Christian Oefinger of TUM's EDGAR team, is reported here as a perspective on where this work goes next – not a closed validation. The full experimental methodology, including whether every compared model received the same adaptation to EDGAR data, has not yet been independently confirmed with TUM, and the result is single-platform; read it accordingly. Concretely:

- **Real-vehicle retraining:** TUM trained AutoE2E from scratch on 134 shards (∼27,000 samples) of its own EDGAR vehicle data, reaching 0.430 m ADE and 1.224 m FDE at 3 s – against a constant-velocity reference of 0.955 m / 2.605 m on the same data – within a few hours of training by the working group's account.
- **Efficiency evidence:** inference on a single A6000 GPU is 101 ms at 0.5 GiB memory. Compared zero-shot on the same EDGAR data, Alpamayo 1.5 reaches ∼0.99 m (segment 1) and ∼0.64 m (segment 2) ADE at 3 s, and SimForge-D1.5 reaches ∼1.00 m and ∼0.76 m across the same two segments, both at roughly 800 ms inference and 14–21 GiB memory – the first evidence that AutoE2E's parameter budget and architecture translate into a real latency and memory advantage on vehicle hardware, not only on the open-loop benchmarks of Sec. 5.

Karlsruhe Institute of Technology (KIT) – Joining the Autoware Foundation

KIT, one of Germany's leading technical universities, has joined the Autoware Foundation as an Academic Member, and its Autonomous Driving Team at the Institute of Measurement and Control Systems (MRT) will collaborate with Autoware to bring full-stack end-to-end driving onto European roads. KIT supports Autoware's Robotaxi Working Group in its efforts towards E2E driving:

- **Independent benchmarking:** KIT is independently benchmarking Autoware's open-loop trained model release on KITScenes-Multimodal – the urban autonomous driving dataset combining high-fidelity sensor data with complete Lanelet2 HD maps.
- **Training and real-world deployment:** KIT will collaborate with Autoware to further train and deploy the E2E model on its test vehicle *Joy* and to demonstrate autonomous driving in Germany.

Autoware E2E on European roads. *Joy* will serve as the main target platform for Autoware E2E in Europe. Having recorded KITScenes in Karlsruhe, Frankfurt, and Sindelfingen, *Joy* takes end-to-end driving onto challenging German inner-city roads – from dataset to closed-loop road deployment with minimal domain gap.

Beyond this paper, the working group's own publication plan continues with a camera-focused follow-up, a tools-and-evaluation paper (in discussion with KIT, which could also give the working group visibility on KIT's own benchmark page), and a fused-model, closed-loop paper once the roadmap items above land.

9 Contributors and Acknowledgments

Contributors are listed in three tiers, following the working group's own contact records: Project Leads, Core Contributors (split between architecture and benchmarking, people may appear in both), and Other Contributions from the Karlsruhe Institute of Technology and TU Munich.

9.1 Project Leads

Muhammad Zain Khawaja: President of the Autoware Foundation, Robotaxi Working Group direction, architecture design, benchmark design and review. Zaynab El Mawas: Co-chair Robotaxi Working Group, research lead, paper preparation and working-group coordination (University of Lille, France).

9.2 Core Contributors

- **Architecture:** Ryota Yamada (Amazon Web Services, Japan) – architecture, training, evaluation, and code and repository maintenance; Gabriela Cordova (Universidad Carlos III de Madrid, Spain); Arseni Lysak (Warsaw University of Technology, Poland); Intisar Chowdhury (SENSYN ROBOTICS, Inc., Japan); Bharat Jayaprakash (University of Minnesota, USA); Apostolos Georgiadis (cubesys GmbH, Germany); Shri Harsha Adapala (University at Buffalo, USA); Jatin V. (Duckietown Robotics, India); Christoph Bank (TU Munich, Germany); and Alejandra Solarte Uscátegui (Tecnocad, Italy).
- **Benchmarking:** Arseni Lysak – AutoE2E/nuPlan on KITScenes; Shri Harsha Adapala – SimForge; Atanasko Boris Mitrev (EXO Service Solutions, North Macedonia) – Qwen-Drive-1.0, Alpamayo 1.5; Hassan Ali Ansari (TRATON GROUP, Germany) – Drive-JEPA; Nishad Sahu (Carnegie Mellon University, USA) – UniAD; Ryota Yamada (Amazon Web Services, Japan) – METEOR.

9.3 Other Contributions

- **TU Munich (TUM):** Christian Oefinger, coordinating the EDGAR team's real-vehicle work.
- **Karlsruhe Institute of Technology (KIT-MRT):** Richard Schwarzkopf, Fabian Immel, and Piotr Spieker, for the KITScenes Multimodal dataset [12].

Acknowledgments

We thank the contributors and the community of the Autoware Foundation Robotaxi Working Group; Ryohsuke Mitsudome, Simon Thompson, Bin Wang, and Shunya Seiya of TierIV for their support and collaboration

10 Reproducibility

The evaluated model configuration identifier is `bevformer_v2_t8_split_navigation_v5`. The camera pathway is initialized from the official BEVFormer V2 R50 T8 checkpoint, loaded with the BEV queries and the row/column positional embeddings resized to 300×200, the camera embeddings matched to the number of views, and the detection and perspective heads omitted; it carries a `NOASSERTION` weight license and

was trained on nuScenes [11] under CC-BY-NC-SA-4.0, and compatibility of redistributing checkpoints derived from it under an Apache-2.0 release has not yet been independently confirmed. KITScenes originates from KIT-MRT/KITScenes-Multimodal on Hugging Face. Of the checkpoints trained for this release, only `kitscenes-epoch-5` (fine-tuned) and `nuplan-epoch-5` (baseline) are released, together with the model card, on Hugging Face at `AutowareFoundation/auto_e2e`. The evaluation, replay, and integration code is released with the main repository at the tag given in the header of this paper. The code is licensed Apache-2.0; the released pretrained weights are subject to the license terms of the datasets used to train them and are not themselves Apache-2.0. AutoE2E's six camera views are `camera_base_front_center` plus the five ring cameras other than `camera_ring_front`. All checkpoints were evaluated via a common evaluation script, `evaluate_reactive_kitscenes_checkpoint` (`source_split="val"`), with a separate adapter script per compared model.

References

[1] Qwen Team. Qwen-Drive-1.0: An Initial Step towards a Vision-Language Foundation Model for Autonomous Driving. arXiv:2609.00111, 2026.

[2] NVIDIA. Alpamayo-R1: Bridging Reasoning and Action Prediction for Generalizable Autonomous Driving in the Long Tail. arXiv:2511.00088, 2025 (also referred to as Alpamayo 1).

[3] Y. Hu, J. Yang, L. Chen, K. Li, C. Sima, X. Zhu, S. Chai, S. Du, T. Lin, W. Wang, L. Lu, X. Jia, Q. Liu, J. Dai, Y. Qiao, and H. Li. Planning-Oriented Autonomous Driving. In *Proc. IEEE/CVF CVPR*, pp. 17853–17862, 2023. arXiv:2212.10156.

[4] Z. Li, W. Wang, H. Li, E. Xie, C. Sima, T. Lu, Y. Qiao, and J. Dai. BEVFormer: Learning Bird's-Eye-View Representation from Multi-camera Images via Spatiotemporal Transformers. In *Computer Vision – ECCV 2022*, LNCS 13669, pp. 1–18, Springer, 2022. doi:10.1007/978-3-031-20077-9_1.

[5] C. Yang, Y. Chen, H. Tian, C. Tao, X. Zhu, Z. Zhang, G. Huang, H. Li, Y. Qiao, L. Lu, J. Zhou, and J. Dai. BEVFormer v2: Adapting Modern Image Backbones to Bird's-Eye-View Recognition via Perspective Supervision. In *Proc. IEEE/CVF CVPR*, pp. 17830–17839, 2023. arXiv:2211.10439.

[6] X. Zhu, W. Su, L. Lu, B. Li, X. Wang, and J. Dai. Deformable DETR: Deformable Transformers for End-to-End Object Detection. In *Proc. ICLR*, 2021. arXiv:2010.04159.

[7] B. Jiang, S. Chen, Q. Xu, B. Liao, J. Chen, H. Zhou, Q. Zhang, W. Liu, C. Huang, and X. Wang. VAD: Vectorized Scene Representation for Efficient Autonomous Driving. In *Proc. IEEE/CVF ICCV*, pp. 8340–8350, 2023. arXiv:2303.12077.

[8] K. Chitta, A. Prakash, B. Jaeger, Z. Yu, K. Renz, and A. Geiger. TransFuser: Imitation with Transformer-Based Sensor Fusion for Autonomous Driving. *IEEE Trans. Pattern Analysis and Machine Intelligence*, 45(11):12878–12895, 2023. doi:10.1109/TPAMI.2022.3200245.

[9] A. Prakash, K. Chitta, and A. Geiger. Multi-Modal Fusion Transformer for End-to-End Autonomous Driving. In *Proc. IEEE/CVF CVPR*, pp. 7077–7087, 2021. arXiv:2104.09224.

[10] H. Caesar, J. Kabzan, K. S. Tan, W. K. Fong, E. Wolff, A. Lang, L. Fletcher, O. Beijbom, and S. Omari. nuPlan: A closed-loop ML-based planning benchmark for autonomous vehicles. *CVPR 2021 Workshop on Autonomous Driving: Perception, Prediction and Planning (ADP3)*, 2021. arXiv:2106.11810.

[11] H. Caesar, V. Bankiti, A. H. Lang, S. Vora, V. E. Liong, Q. Xu, A. Krishnan, Y. Pan, G. Baldan, and O. Beijbom. nuScenes: A Multimodal Dataset for Autonomous Driving. In *Proc. IEEE/CVF CVPR*, pp. 11621–11631, 2020.

[12] R. Schwarzkopf, F. Immel, A. Blumberg, J. Merkert, N. Rack, K. Wang, F. Konstantinidis, J. Truetsch, C. Fernandez, A. Bätz, K. Rösch, M. Steiner, W. Poh, Y. Shen, R. Wagner, F. Hauser, D. Strutz, J. Villa, G. Stepanov, H. Caesar, Ö. Ş. Taş, F. Bieder, J.-H. Pauls, and C. Stiller. The Road Ahead in Autonomous Driving: The KITScenes Multimodal Dataset. arXiv:2606.02956, 2026.

[13] F. Poggenhans, J.-H. Pauls, J. Janosovits, S. Orf, M. Naumann, F. Kuhnt, and M. Mayr. Lanelet2: A High-Definition Map Framework for the Future of Automated Driving. In *Proc. 21st IEEE ITSC*, pp. 1672–1679, 2018. doi:10.1109/ITSC.2018.8569929.

[14] M. Bansal, A. Krizhevsky, and A. Ogale. ChauffeurNet: Learning to Drive by Imitating the Best and Synthesizing the Worst. In *Proc. Robotics: Science and Systems (RSS)*, 2019. doi:10.15607/RSS.2019.XV.031.

[15] J. Gao, C. Sun, H. Zhao, Y. Shen, D. Anguelov, C. Li, and C. Schmid. VectorNet: Encoding HD Maps and Agent Dynamics From Vectorized Representation. In *Proc. IEEE/CVF CVPR*, pp. 11525–11533, 2020.

[16] M. Liang, B. Yang, R. Hu, Y. Chen, R. Liao, S. Feng, and R. Urtasun. Learning Lane Graph Representations for Motion Forecasting. In *Computer Vision – ECCV 2020*, LNCS 12347, pp. 541–556, Springer, 2020. doi:10.1007/978-3-030-58536-5_32.

[17] J.-T. Zhai, Z. Feng, J. Du, Y. Mao, J.-J. Liu, Z. Tan, Y. Zhang, X. Ye, and J. Wang. Rethinking the Open-Loop Evaluation of End-to-End Autonomous Driving in nuScenes. arXiv:2305.10430, 2023.

[18] Z. Li, Z. Yu, S. Lan, J. Li, J. Kautz, T. Lu, and J. M. Alvarez. Is Ego Status All You Need for Open-Loop End-to-End Autonomous Driving? In *Proc. IEEE/CVF CVPR*, pp. 14864–14873, 2024. arXiv:2312.03031.

[19] D. Dauner, M. Hallgarten, A. Geiger, and K. Chitta. Parting with Misconceptions about Learning-based Vehicle Motion Planning. In *Proc. 7th Conference on Robot Learning (CoRL)*, PMLR 229, pp. 1268–1281, 2023.

[20] D. Dauner, M. Hallgarten, T. Li, X. Weng, Z. Huang, Z. Yang, H. Li, I. Gilitschenski, B. Ivanovic, M. Pavone, A. Geiger, and K. Chitta. NAVSIM: Data-Driven Non-Reactive Autonomous Vehicle Simulation and Benchmarking. In *Advances in Neural Information Processing Systems 37 (NeurIPS 2024), Datasets and Benchmarks Track*, 2024. arXiv:2406.15349.

[21] B. Jaeger, K. Chitta, and A. Geiger. Hidden Biases of End-to-End Driving Models. In *Proc. IEEE/CVF ICCV*, pp. 8240–8249, 2023. arXiv:2306.07957.

[22] K. He, X. Zhang, S. Ren, and J. Sun. Deep Residual Learning for Image Recognition. In *Proc. IEEE CVPR*, pp. 770–778, 2016.